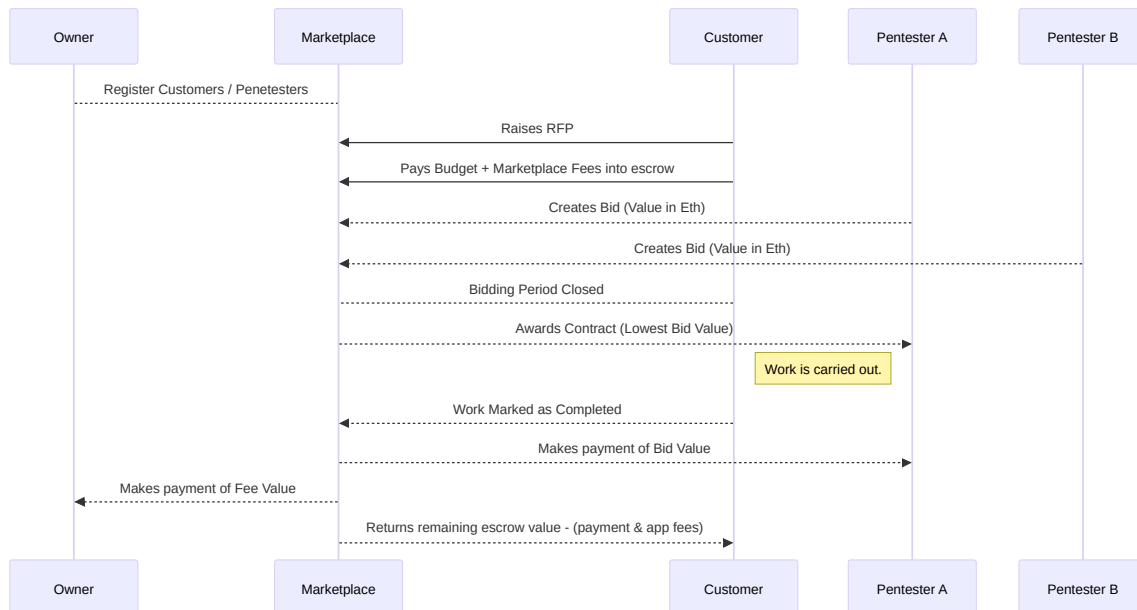


Application Flow Demo

The following shows a run through of how the application works, using the following example flow to demonstrate functionality.



Step 0 - Deploy the Contract within the Remix IDE

The "Reporting" contract should be compiled and deployed to the Remix IDE EVM using the first account in the list. This will become the owner:

Owner - 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4

Step 1 - Register Customers and Pentesters (Owner)

The following accounts were used within Remix:

Customers - 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2 -
0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db

Pentesters - 0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB -
0x617F2E2fD72FD9D5503197092aC168c91465E7f2

Add Customers:

Balance: 0 ETH

ADDCUSTOMER

_address: 0xAb8483F64d9C6d1EcF9b849Ae677

_name: Customer One

Calldata Parameters transact

Balance: 0 ETH

ADDCUSTOMER

_address: 0x4B20993Bc481177ec7E8f571ceCaE

_name: Customer Two

Calldata Parameters transact

Add Pentesters:

ADDPENTESTER

_address: 0x78731D3Ca6b7E34aC0F824c42a7c

_name: Pentester A

Calldata Parameters transact

ADDPENTESTER

_address: 0x617F2E2fD72FD9D5503197092aC1

_name: Pentester B

Calldata Parameters transact

Step 2 - Customer Raises RFP and Pays Budget into Escrow

In Remix, switch to the first customer account, and then choose CreateRFP.

Field	Value
Max Budget	The total budget for this RFP contract, in Wei.
Bidding End Datetime	A Unix timestamp in the future.
Title	The title for this piece of work.
RFP Doc URL	The URL for the fictional RFP document within the webapp.

Note: the Customer must send the value of Max Budget + the current fee with this transaction. The current fee percentage can be found using the ShowFee function:

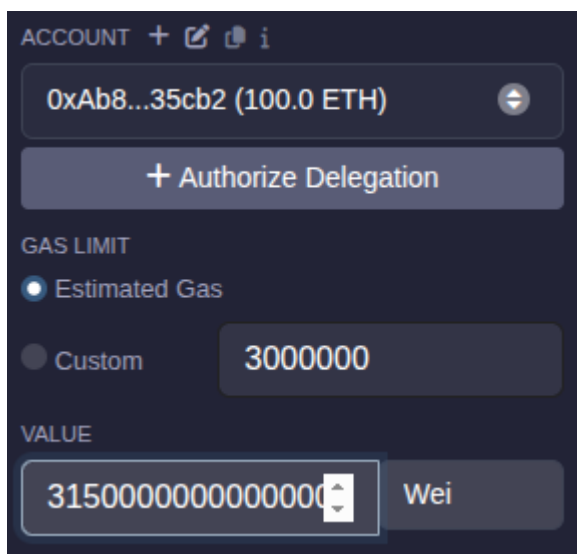


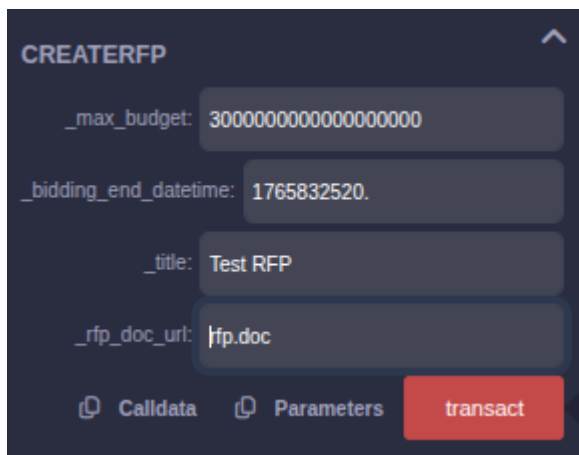
This document is being written at 2055 on 15/12/2025, so the timestamp will be set to expire at 2102, which is 1765832520.

The minimum value is 1 Eth by default, so the Customer must send 1.05 Eth (or 105000000000000000 Wei) for every Eth in Max Budget.

The value of this RFP contract is 3 Eth, so the values are:

- Max Budget: 3 Eth
- Fee: 0.15 Eth
- To be sent: 315000000000000000 Wei





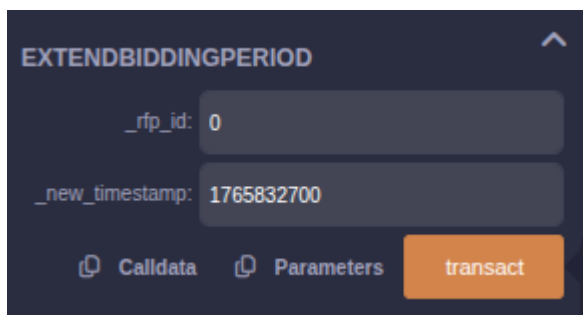
A screenshot of a web form titled "CREATERFP" with a small upward arrow icon in the top right corner. The form contains four input fields: "_max_budget:" with the value "3000000000000000000", "_bidding_end_datetime:" with the value "1765832520.", "_title:" with the value "Test RFP", and "_rfp_doc_url:" with the value "rfp.doc". At the bottom of the form, there are three buttons: "Calldata" and "Parameters" with document icons, and a red "transact" button.

This website is useful for conversions between fractions of an Eth: <https://eth-converter.com/>.

This one is useful for finding a Unix epoch value from a date time: <https://www.epochconverter.com/>

The balance of the Smart Contract is now 3.15 Eth.

The customer can extend the bidding period if they have received no bids, in this instance to 2105 (1765832700).



A screenshot of a web form titled "EXTENDBIDDINGPERIOD" with a small upward arrow icon in the top right corner. The form contains two input fields: "_rfp_id:" with the value "0" and "_new_timestamp:" with the value "1765832700". At the bottom of the form, there are three buttons: "Calldata" and "Parameters" with document icons, and an orange "transact" button.

Step 3 - Pentesters Bid

The pentesters need to bid on the contract. Switching to the account of Pentester A, who bids 2 Eth:

MAKEBID

_rfp_id:

0

_bid_value:

20000000000000000000

_response_doc_url:

ResponseA.doc

Calldata

Parameters

transact

And switching to Pentester B who then bids 2.5 Eth:

MAKEBID

_rfp_id:

0

_bid_value:

25000000000000000000

_response_doc_url:

ResponseB.doc

Calldata

Parameters

transact

Provided it is before the end of the bidding phase, the pentesters can adjust their bidding. Note that there is only one bid per Pentester. The previous bid is updated with the new value.

Until the Bidding Period expires, there is nothing more that must be done by the Pentesters once they are happy with their bids. They are, however, able to view their Active Bids and can View All Bids, for example:

Pentester A - 2.0 Eth:

```
0: tuple(uint256,string,uint256,uint256,address,bool,uint8[]): 0,Test RFP,2000
0000000000000000,0,0x000000000000
00000000000000000000000000000000,f
      else,0
```

[viewAllMyBids](#)

viewAllMyBids - call

```
0: tuple(uint256,string,uint256,uint256,address,bool,uint8)[]: 0,Test RFP,2000
0000000000000000,0,0x000000000000
00000000000000000000000000000000,f
      else,0
```

Pentester B - 2.5 Eth:

[viewActiveBids](#)

viewActiveBids - call

```
0: tuple(uint256,string,uint256,uint256,address,bool,uint8)[]: 0,Test RFP,2500
0000000000000000,0,0x000000000000
00000000000000000000000000000000,f
      else,0
```

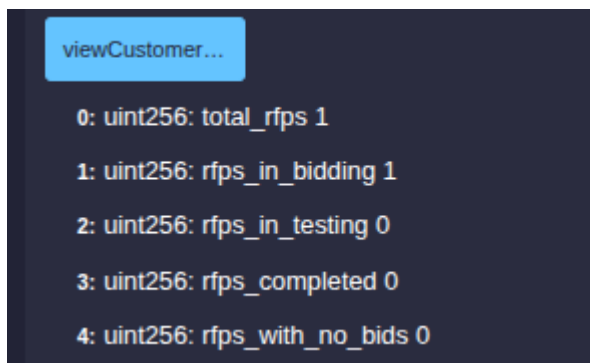
[viewAllMyBids](#)

```
0: tuple(uint256,string,uint256,uint256,address,bool,uint8)[]: 0,Test RFP,2500
0000000000000000,0,0x000000000000
00000000000000000000000000000000,f
      else,0
```

The customer can also look at a list of their RFPs and the current state of them:

[viewOwnRFPs](#)

```
0: tuple(address,address,uint256,uint256,uint256,uint256,string,string,uint8)[]: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2,0x000000000000000000000000000000000000,3000000000000000000,0,1765832700,2,2,Test RFP,rfp.doc,0
```



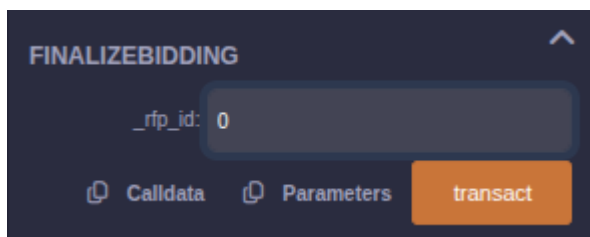
Step 4 - Bidding Period Closed

There is no such thing as a timer or internal cron task within the Ethereum blockchain; Smart Contracts do not have automatic triggers. Therefore, the code checks to see whether the Bidding Period has expired every time a new bid is made, or when the Customer tries to extend the bidding time.

We envisage that an external web app would be used to drive the application functions. As well as creating a new RFP, this would also set up an external timer function - such as a Unix cron job - that would be used to call the Finalize Bidding function just after the Bidding Time expires.

This would ensure that the functions to award the winning bidder the contract for the work are run in a timely fashion.

Only the contract owner can call this function.



Step 5 - Lowest Bidder Awarded as Winner

The tendering process awards the winning bid status to the lowest bidder.

The Customer can see that their RFP has moved into the 'In Testing' phase, meaning that actual testing work is being done by the pentesters. This could take any length of time from a few days to many months to complete.

viewCustomer...

```
0: uint256: total_rfps 1
1: uint256: rfps_in_bidding 0
2: uint256: rfps_in_testing 1
3: uint256: rfps_completed 0
4: uint256: rfps_with_no_bids 0
```

The pentesters can see their winning and losing bids, for example Pentester A (who won this RFP):

Winning:

viewWinningB...

```
0: tuple(uint256,string,uint256,uint256,address,bool,uint8[]): 0,Test RFP,2000000000000000000,20000000000000000000000,0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB,true,1
```

Losing (none):

viewLosingBids

```
0: tuple(uint256,string,uint256,uint256,address,bool,uint8[]):
```

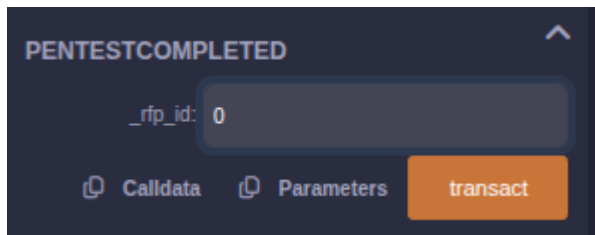
A pentester can also get a summary, which would be displayed more neatly in the Web App:

viewPentester...

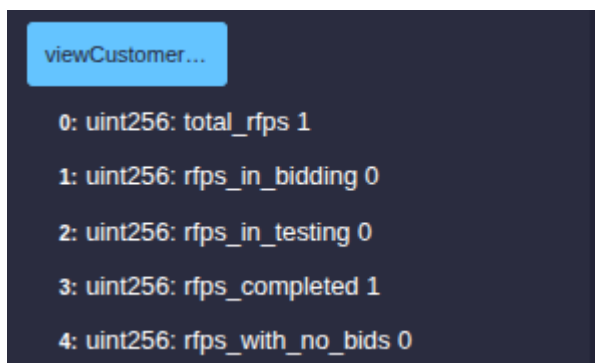
```
0: uint256: total_earnings 0
1: uint256: total_wins 1
2: uint256: total_bids 1
3: uint256: pending_payments 2000000000000000000
```

Step 6 - Work Marked as Completed

When the Customer is happy that the work has been completed by the Pentester, they can change this state to initiate a payment.



The state is changed:



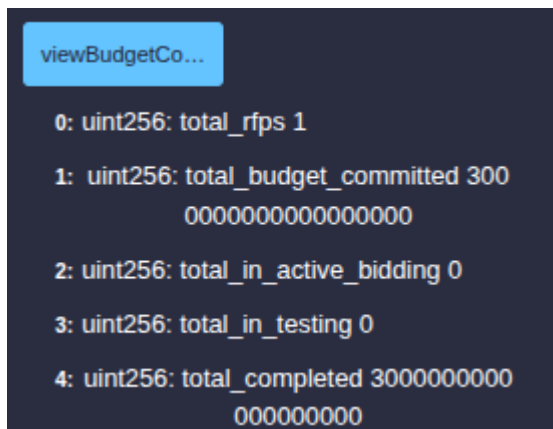
At this point, payments are made to each of the parties:

- Pentester is paid their Bid Value.
- Smart Contract Owner is paid their 5% fee.
- Customer is returned any remaining balance.

The value in the Smart Contract is now 0 Eth again, as this is the only RFP being managed at this time.



The customer can see their marketplace statistics and budget commitments:



The Smart Contract Owner can also get statistics:

